RAPPORT IPO

I. General Informations

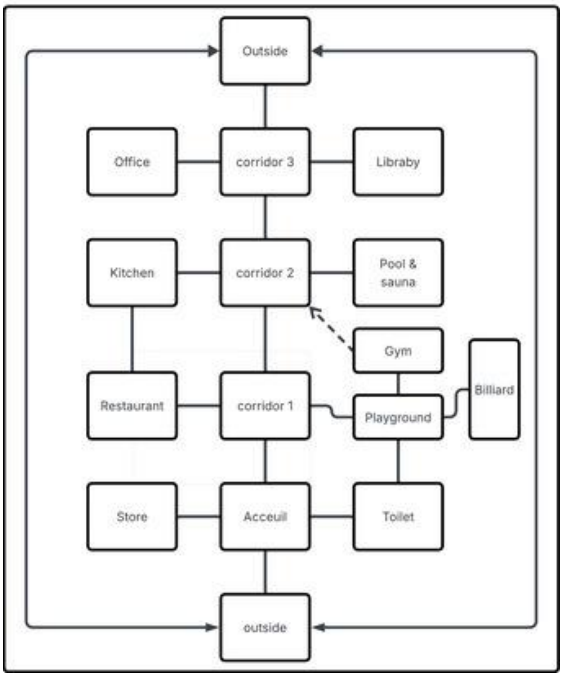
I.A) Auteurs: Minh Quang TRAN

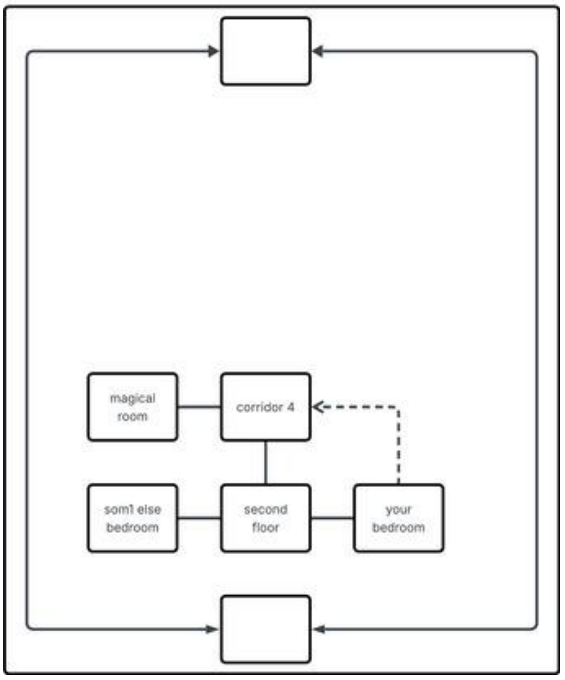
I.B) In a hotel, a spy realized that a group of terrorist have place multiple bomb on the hotel

I.C) Résumé du scénario (complet)

I.D) Plan

- On the left - ground floor, on the right - first floor





I.E) Scénario détaillé (complet, avec indication de la partie "réduit" si exercice 7.3.3)

I.F) Détail des lieux, items, personnages

Lieux

- corridor
- kitchen
- restaurant
- playground
- gym
- billiard
- library
- piscine et sauna

- toilet
- boutique

items

```
//create items
        Item umbrella = new Item("umbrella", "blue and light", 2);
        Item lightBulb = new Item("light bulb", "from the ceiling", 1);
        Item billiardStick = new Item("billiard stick", "wooden stick", 3);
        Item oil = new Item("oil", "a small can with 50ml cooking oil", 2);
        Item ladle = new Item("ladle", "steel cooking ladle", 2);
        Item broom = new Item("broom", "used daily to clean the toilet", 2);
        Item weight = new Item("weight", "10kg weight", 4);
        Item towel = new Item("towel", "free to use all the pool", 2);
        Item pepperShaker = new Item("peppershaker", "small and portable", 1);
        Item napkin = new Item("napkin", "white", 0);
        Item plasticBall = new Item("plastic ball", "bouncy and fun to play
with", 1);
        Item ballPointPen = new Item("ball point pen", "blue", 1);
        Item screwdriver = new Item("screwdriver", "slotted with magnet
attached", 2);
```

personnages

-
- to be added

I.G) Situations gagnantes et perdantes

- après désamorcer tous les bombes le joueur gagne
- les bombes sont cachés dans n'importe quel pièces
- il faut collecter les items et résoudre des mini jeux pour désamorcer des bombes

I.H) Éventuellement énigmes, mini-jeux, combats, etc.

I.I) Commentaires (ce qui manque, reste à faire, ...)

II. Réponses aux exercices

Exercise 7.5: PrintLocationInto

```
private void printLocationInfo()
    {
        System.out.println( "You are " + currentRoom.getDescription());
        System.out.print("Exits: ");
        if(currentRoom.northExit != null) {
            System.out.print("north ");
        }
        if(currentRoom.eastExit != null) {
            System.out.print("east ");
        }
        if(currentRoom.southExit != null) {
            System.out.print("south ");
        }
        if(currentRoom.westExit != null) {
            System.out.print("west ");
        }
    }
```

Print the current location information of the room, with its possible exits

In the future other method can use this function

Exercise 7.6: getExit

method goRoom

```
/**
     * if the player only enter go without second word, print go where
     * if there is no room to go, print possible exits
     * otherwise the player go to the next room
     */
    private void goRoom(final Command pCommand){
        if(!pCommand.hasSecondWord()){
            this.aGui.println("go where?");
            return;
        }else{
            String vDirection = pCommand.getSecondWord();
            Room vNextRoom = this.currentRoom.getExit(vDirection);
            if(vNextRoom == null){
                System.out.println(vDirection);
                System.out.println("There is no door !");
                System.out.println(currentRoom.getExitString());
                return;
            }else{
                System.out.println("currently " + currentRoom.getDescription());
            }
        }
    }//goRoom
```

```
if(!pCommand.hasSecondWord()){
    System.out.println("go where");
    return;
}else{
    Room vNextRoom = null;
    String vDirection = pCommand.getSecondWord();

    vNextRoom = this.currentRoom.getExit(vDirection);
```

```
private String description;
private Room northExit;
private Room southExit;
private Room eastExit;
private Room westExit;

    public Room getExit(String direction)
    {
        if (direction.equals("north")) {return northExit;}
        if(direction.equals("east")) {return eastExit;}
        if(direction.equals("south")) {return southExit;}
        if (direction.equals("west")) {return westExit;}
        return null;
    }//getExit
```

Exercise 7.7: getExitString

getExitString

```java
/**
     * checking each exit to see which one is available,
     * the one that is available is then added to the string
     * to be returned
     */
    public String getExitString(){
        String exitString = "Exits: ";
        if (northExit != null){exitString += "north ";}
        if (eastExit != null){exitString += "east ";}
        if(southExit != null){exitString += "south ";}
        if (westExit != null){exitString += "west ";}
        return exitString;
    }//old getExistString
```

```java
if(vNextRoom == null){
    System.out.println(vDirection);
    System.out.println("There is no door !");
    System.out.println("currently in " + currentRoom.getDescription());
    System.out.println(currentRoom.getExitString());
    return;
}else{
    this.currentRoom = vNextRoom;
    System.out.println("currently in " + currentRoom.getDescription());
}
```

qzqz

Exercise 7.8: Hashmap, setExits

HashMap

```
import java.util.HashMap;
import java.util.Set;
private HashMap<String, Room> exits; //store exits of a room
/**
     * @hashmap
     */
    public Room(final String pDescription)
    {
        this.aDescription = pDescription;
        this.exits = new HashMap<String, Room>();
    }//Room
```

setExit: set initial exits

```
/**
     * @set initial exits
     */
    public void setExits(Room north, Room east, Room south, Room west){
        if(north != null){exits.put("north", north);}
        if(east != null){exits.put("east", east);}
        if(south != null){exits.put("south", south);}
        if(west != null){exits.put("west", west);}
    }//*Room*.setExit("*direction*", *AnotherRoom*);
    /**
     * @set single exit for a new room (location)
     */
    public void setExit(String direction, Room r){
        exits.put(direction, r);
    }//setExit
```

New room added

```
Room secondfloor = new Room("at the second floor", "secondfloor");

acceuil.setExit("up", secondfloor);
secondfloor.setExit("down", accueil);
```

Exercise 7.9: keySet

```
/**
     * return all of the possible exit of a room
     * @avoid coupling in getLocationInfo and no exit
     */
    public String getExitString(){
        String sExit = "Exits: ";
        Set<String> keys = exits.keySet();
        for(String exit: keys){
            sExit += exit + " ";
        }
        return sExit;
```

Exercise 7.10: getExitString

exits.keySet() is a set of keys of HashMap, we declared a variable keys, which goes through every exit to see which one is possible.

for(String exit: keys) serve as a for-each loop

the String sExit will add the one that is useable (and a space “ “)

Exercise 7.11: getLongDescription

```
/**
     * print the current info, including the current location, items and exit
     * once we add items or monster we only need to change this class
     */
    public String getLongDescription(){
        return "You are " + this.aDescription + '\n' + getExitString();
    }//getLongDescription
```

Exercise 7.14, 7.15: look & eat

- 2 new commands “look” and “eat” are added into the aValidCommand String.

```
private final String[] aValidCommands = {"go", "look", "quit", "eat", "look"};
```

2 new instructions added in “processCommand”

```
private boolean processCommand(final Command pCommand){
        if(pCommand.isUnknown()){
            System.out.println("i don't know what you mean...");
            System.out.println("what you wrote: " + pCommand.getCommandWord());
            return false;
        }else if(pCommand.getCommandWord().equals("quit")){
            this.aGui.println("You just quitted the game");
            return true;
        }else if(pCommand.getCommandWord().equals("go")){
            this.goRoom(pCommand);
            return false;
/**/    }else if(pCommand.getCommandWord().equals("look")){
/**/         look();
/**/         return false;
/**/     }else if(pCommand.getCommandWord().equals("eat")){
/**/         eat();
/**/         return false;
        }else if(pCommand.getCommandWord().equals("help")){
            help();
            return false;
        }else{
            return false;
        }
    }//processCommand
```

procédure look() and eat()

```
/**
     * print the current info, including location and exit
     * once we add items and monster we only need to change the classe in Room
     */
    private void look(){
        System.out.println(this.currentRoom.getLongDescription());

    }//look
    private void eat(){
        System.out.println("You have eaten and you are not hungry any more");
    }//eat
```

Exercise 7.16: showAll, showCommands

Class: CommandWords

```
/**
 *print valid words
 */
public void showAll(){
        for(String command: aValidCommands){
            System.out.println(command + " ");
        }
}
```

Class: CommandWords

```
public void showCommands(){
     aValidCW.showAll();
}//showCommands
```

Class: Game

```
   /**
     * print help
     */
    private void help(){
        System.out.println("You are lost");
        System.out.println("You command words are: ");
        System.out.println(aParser.showCommands);
    }//help
```

Exercise 7.17: changer Game?

If we add a new command, the new valid word has to be added in the CommandWords class and still, we need to change a little bit in the game class. We have to add that new word (command) into the processCommand and call another method. However we don't need to change the help command (and any method that use getExitString) as it automatically included the new command word

Exercice 7.18: getCommandList

Class CommandWord:

```
    /**
      * return all valid words without printing it like "showAll" procedure
      */
     public String getCommandList(){
         String allCommand = "";
         for(String command: aValidCommands){
             allCommand += command + " ";
         }
         return allCommand;
     }//getCommandList
```

Class Parser:

```
public void showCommands(){
        aValidCW.getCommandList();
    }//showCommands
```

Class CommandWord:

```
    /**
      * print help
      */
     private void help(){
         System.out.println("You are lost");
         System.out.println("You command words are: ");
         System.out.println(aParser.showCommands);
     }//help
```

Exercice 7.18.6

Command & CommandWord: no change

Room:

- add image ⇒ modify constructeur ⇒ now each Room have one more characteristic (or paramètre) which associate with an image (from the file)

Parser

- We remove scanner because there is no direct input (Interface graphique)
- replace scanner with Tokenizer which cut the input String into words

GameEngine

- System.out.print ⇒ this.aGui.println for the interface graphique
- quit() ⇒ endGame()
- processCommand() ⇒ InterpretCommand(), a procédure so need to be return

UserInterface:

- affiche what the user see, including the button, the image and entryfield ⇒ can also be modified

Exercice 7.18.8: Button

javax. necessary imported

```
//import javax.swing.;
import javax.swing.JFrame;
import javax.swing.JTextField;
import javax.swing.JTextArea;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.ImageIcon;
import javax.swing.JButton;
//import java.awt.;
import java.awt.Dimension;
import java.awt.BorderLayout;
//import java.awt.event.;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.awt.event.ActionEvent;
```

2 buttons declared

```
private JButton     aQuitButton;
private JButton     aLookButton;
```

Things added onto createGui

```
// text appeared on the button
this.aQuitButton = new JButton ("Quit");
this.aLookButton = new JButton ("Look");
// button position
vPanel.add( this.aQuitButton, BorderLayout.EAST );
vPanel.add( this.aLookButton, BorderLayout.WEST );
// add some event listeners to some components
this.aQuitButton.addActionListener( this );
this.aLookButton.addActionListener( this );
```

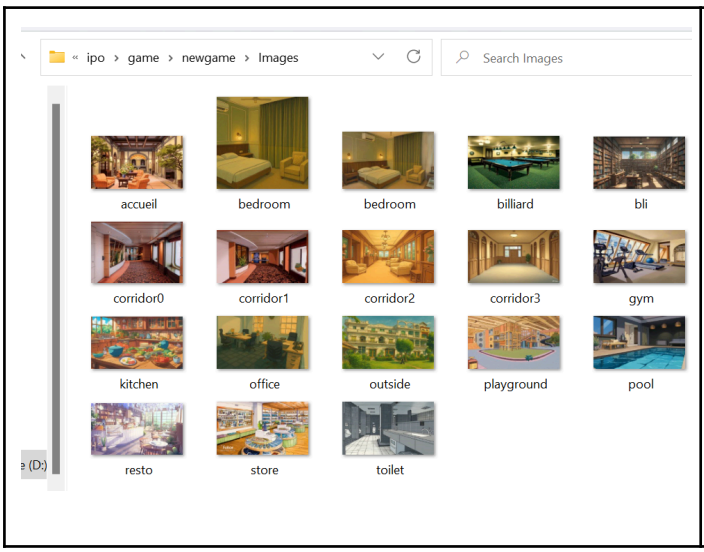
@Override actionPerformed

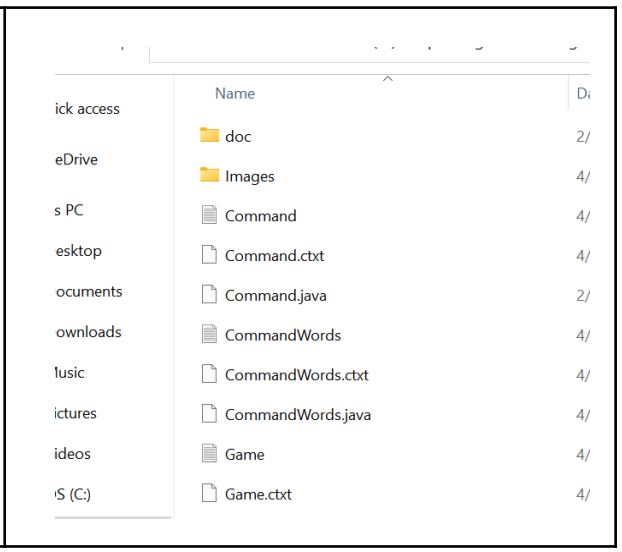
```
//Actionlistener interface for entry textfield.
    @Override public void actionPerformed( final ActionEvent pE )
    {
        // no need to check the type of action at the moment
        // because there is only one possible action (text input) :
        if ( pE.getSource() == aQuitButton ){
            aEngine.interpretCommand("quit");
        }else if ( pE.getSource() == aLookButton ){
            aEngine.interpretCommand("look");
        }else if ( pE.getSource() == aEntryField ){
            this.processCommand(); // never suppress this line
        }
    } // actionPerformed(.)
```

Exercice 7.19.2: folder Images

new folder Items created, ImageName of each room is changed to each match the image

```
Room outside = new Room("outside", "Images/outside.jpeg");
Room accueil = new Room("at the accueil", "Images/accueil.jpg");
Room corridor1 = new Room("at corridor1", "Images/corridor1.jpg");
Room corridor2 = new Room("at corridor2", "Images/corridor2.jpg");
Room corridor3 = new Room("at corridor3", "Images/corridor3.jpg");
Room boutique = new Room("at the store", "Images/store.jpg");
Room toilet = new Room("in the toilet", "Images/toilet.jpeg");
Room restaurant = new Room("in the restaurant", "Images/resto.jpg");
Room pool = new Room("at the swimming pool and sauna", "Images/pool.jpg");
Room kitchen = new Room("in the kitchen", "Images/kitchen.jpg");
Room kid = new Room("at the kids playground", "Images/playground.jpg");
Room gym = new Room("in the gym", "Images/gym.jpg");
Room billard = new Room("in the billiard", "Images/billiard.jpg");
Room office = new Room("at the office", "Images/office.jpg");
Room library = new Room("at the library", "Images/bli.jpg");
```





Exercice 7.20 & 7.21: Item

```
public class Item
{
    private String          aItemNom;
    private String          aItemDescription;
    private int             aItemWeight;
    /**
     * constructeur naturel
     * description and weight of the item
     */
    public Item(final String pItemNom, final String pItemDescription, final int
pItemWeight){
        this.aItemNom = pItemNom;
        this.aItemDescription = pItemDescription;
        this.aItemWeight = pItemWeight;
    }

    /**
     * 3 function, one return the name, one return the descreption and the other one
return the weight
     */
    public String getItemNom(){return this.aItemNom;}//getitem nom
    public String getItemDescription(){return this.aItemDescription;}//getitem
description
    public int getItemWeight(){return this.aItemWeight;}//getitemweight

    /**
     * return the name, description and weight of the item
     */
    public String getItemLongDescription()
    {
        return this.aItemNom + " : " + this.getItemDescription() + " - weight : " +
this.getItemWeight();
    }//getItemLongDescription.
}
```

Class Room

Constructeur

```
private Item        aItem;
private HashMap<String, Item> items; //store items of a room
```

public Room:

```
public Room(final String pDescription, final String pImage)
    {
        this.aDescription = pDescription;
        this.exits = new HashMap<String, Room>();
        /**/this.items = new HashMap<String, Item>();
        this.aImageName = pImage;
    }//Room
```

The items is added the same way with the exits

Exercice 7.22: Items

Class Room

- addItem (I only put the name and weight because the description is long)

```
/**
     * add and Item to the room
     * however it uses only one paramètre because the string s (that I declared)
     * can give the description of the item based on the input item
     */
    public void addItem(Item item){
        String s = item.getItemNom() + " - " + item.getItemWeight();
        items.put(s, item);
    }//addItem
```

getItemString

```
/**
     * return all the item in a room (with its weight)
     */
    public String getItemString(){
        String sItem = "Items: {";
        Set<String> keys = items.keySet();
        for(String item: keys){
            sItem += "(" + item + ") ";
        }
        if(sItem.equals("Items: {")){
            return "No item found!";
        }
        return sItem + "}";
    }//getExitString
```

```
>
> look
You are at the acceuil
Exits: east south north west
Items: {(light bulb - 1) (umbrella - 2) }
```

//orthographe problem has been fixed

getLongDescription

```
/**
     * print the current info, including the current location, items and exit
     * once we add items and monster we only need to change this class
     */
    public String getLongDescription(){
        return "You are " + this.aDescription + '\n' + getExitString() + '\n' +
getItemString();
    }//getLongDescription
```

Class GameEngine, constructeur

```
//create items
        Item umbrella = new Item("umbrella", "blue and light", 2);
        Item lightBulb = new Item("light bulb", "from the ceiling", 1);
        Item billiardStick = new Item("billiard stick", "wooden stick", 3);
        Item oil = new Item("oil", "a small can with 50ml cooking oil", 2);
        Item ladle = new Item("ladle", "steel cooking ladle", 2);
//put item into rooms
        acceuil.addItem(umbrella);
        acceuil.addItem(lightBulb);
        billard.addItem(billiardStick);
        kitchen.addItem(oil);
        kitchen.addItem(ladle);
```

(there is still more on the code, I don't paste it here because it's long)

Exercice 7.23, 7.24 & 7.25: Back

This is my old code, this one only allows the player to back once, but not multiple times.

New word command “back” added

Class CommandWords:

```
private final String[] aValidCommands = { "go", "look", "quit", "eat", "look",
"back"};
```

Class GameEngine

- new command in InterpretCommand
- Print the sentence in quote if the commandString has two word (because “back” command only has one word)

```
else if( vCommandWord.equals("back")){
            if (vCommand.hasSecondWord() )
                this.aGui.println("back command does not have a second word!");
            else
                this.back();
}
```

/**

7.24 and 7.25

the back was working good but works only one time

*/

A new room “preRoom” is declared to memorise the previous room (after this.currentRoom = vNextRoom)

```
private Room            preRoom = null;
```

In the procédure goRoom, if vNextRoom == null, we ignore it, otherwise we use preRoom to memorise the current room before switching to the next one (three line I added are indicated after `/**/`)

```
private void goRoom(final Command pCommand){
        if(!pCommand.hasSecondWord()){
            this.aGui.println("go where?");
            return;
        }else{
            String vDirection = pCommand.getSecondWord();
            Room vNextRoom = this.currentRoom.getExit(vDirection);
            if(vNextRoom == null){
                this.aGui.println(vDirection);
                this.aGui.println("There is no door !");
                this.aGui.println("currently " + currentRoom.getDescription());
                this.aGui.println(currentRoom.getExitString());
                return;
            }else{
                /**/Room memoRoom = currentRoom;
                /**/this.currentRoom = vNextRoom;
                /**/preRoom = memoRoom;
                //this.aGui.println( this.currentRoom.getLongDescription() );
                this.aGui.println("currently " + currentRoom.getDescription());
                if ( this.currentRoom.getImageName() != null )
                this.aGui.showImage( this.currentRoom.getImageName() );
            }
        }
    }//goRoom
```

Procédure back:

```
private void back(){
        if(preRoom == null){
            this.aGui.println("you just started the game");
        }else{
            this.currentRoom = preRoom;
            //this.look();
            this.aGui.println("currently " + currentRoom.getDescription());
            if ( this.currentRoom.getImageName() != null){
                this.aGui.showImage( this.currentRoom.getImageName() );
            }
        }
    }//back
```

Exercice 7.26: Back and Stack

New import and attribute (my room “preRoom” become a stack)

```
import java.util.Stack;
private Stack<Room>     preRoom = new Stack<>();
```

- The stack “preRoom” hasn’t been initialised ⇒ there is no room in the stack when the game starts.

- Whenever the player want to go a new room, the one that he is currently in is added to the stack (`this.preRoom.push(currentRoom)`), before he actually goes to the next one (`this.currentRoom = vNextRoom`)

- Whenever he uses the “back”, he goes to the last room on the stack (`this.preRoom.peek()`). And that room will be suppressed from the stack (`this.preRoom.pop()`).

- When the player “back” to the start, the stack is empty (`preRoom.empty()`), and the next “back” command will print out `"you just went back to the start"`

procédure goRoom:

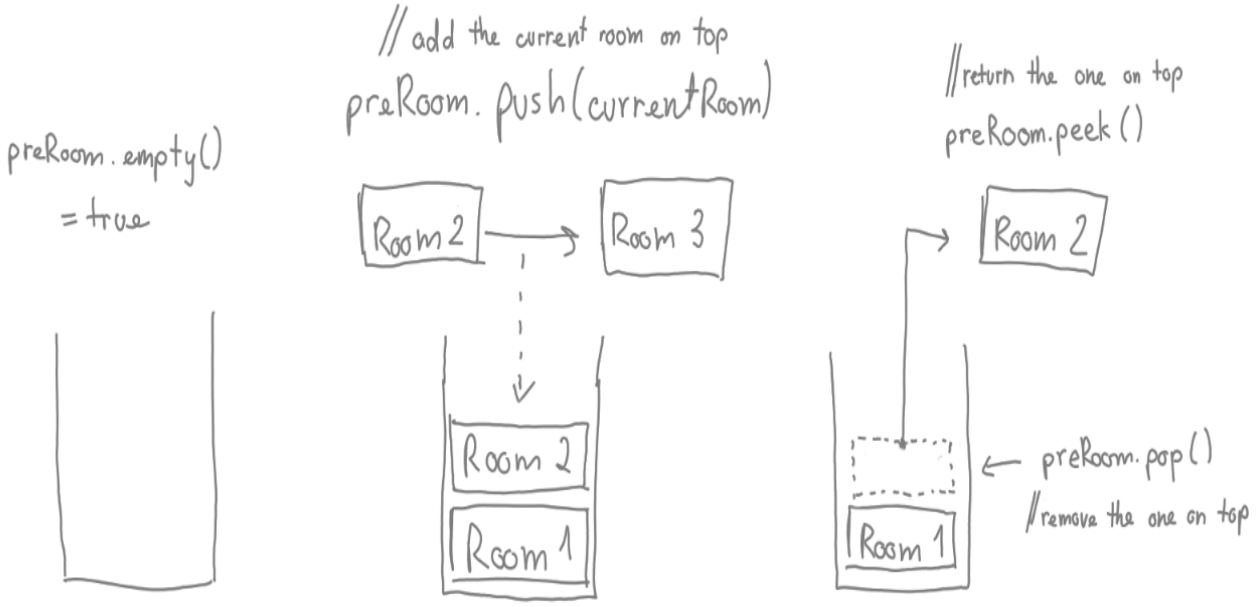
```
this.preRoom.push(currentRoom);
this.currentRoom = vNextRoom;
```

procédure back:

```
/**
     * back allow user to go back to the previous room, can be stacked
     */
    private void back(){
        if(preRoom.empty()){//preRoom.empty() c.à.d the player has went back to
the start
            this.aGui.println("you just went back to the start");
        }else{
            this.currentRoom = this.preRoom.peek();
            this.preRoom.pop();

            this.aGui.println("currently " + currentRoom.getDescription());
            if ( this.currentRoom.getImageName() != null){
```

```
                this.aGui.showImage( this.currentRoom.getImageName() );
            }
        }
    }//back
```



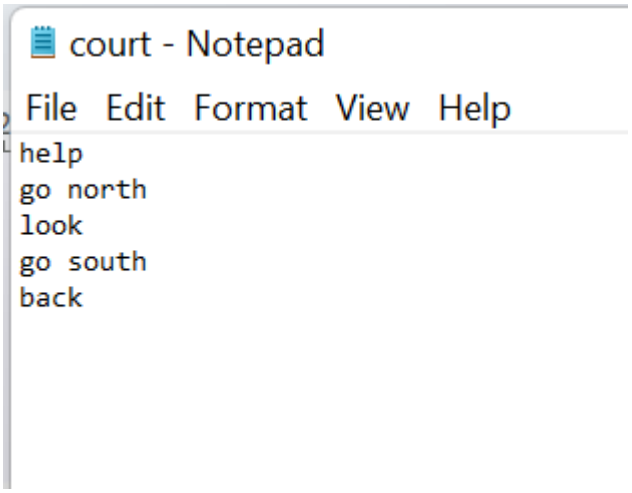
Exercice 7.28: command test

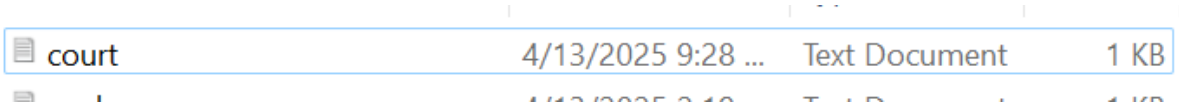
Class GameEngine:

- the **Scanner** is going to scan each line of the text file, if there is a command in following line (**.hasNextLine()**), **vCommandline** will be use to stored that command (**.NextLine()**) and to be put in to **interpretCommand()**
- **try{}** is used to test the command in the file. Whenever there is something goes wrong with the file name or with the command on the file, the code on **catch{}** will be execute
- **finally{}** is used to let me know that the try or catch is done (unnecessary for the code to work, however, when we keep writing commands, we lost track of when the try{} ended)

```
/**
     * testCommand
     * @param pCommand
     */
    private void testCommand(final Command pCommand){
        if(!pCommand.hasSecondWord()){
            this.aGui.println("this command require a second word");
        }else{
            String textFileName = "test/" + pCommand.getSecondWord() + ".txt";
            // pCommand.getSecondWord() si the second word of the user's input
            // which is also the name of the text file to be tested
            // the "test/" and ".txt" guide the system to the file location
            String vCommandLine;
            //the commands of the player on the text file
            Scanner vScanner;
            try{
                vScanner = new Scanner(new File(textFileName));
                while(vScanner.hasNextLine()){
                    vCommandLine = vScanner.nextLine();
                    //this.aGui.println(vCommandLine);
                    this.interpretCommand(vCommandLine);
                }
            }catch(Exception e){
                this.aGui.println("the file with that name doesn't exit");
            }finally{
                this.aGui.println("text file have been tested");
            }
        }
    }
```

Text file “court.txt”





More test file have been added

Exercice 7.29: New class Player

Class GameEngine

- new object aPlayer added to GameEngine

```
private Player          aPlayer;

    /**
     * start the game
     */
    public GameEngine(){
        /**/this.aPlayer = new Player();
        //declare before createRooms to avoid nullPointException/
        this.createRooms();
        this.aParser = new Parser();
    }//game
```

Information stored

1. aCurrentRoom

- **aCurrentRoom** is moved from class GameEngine to class Player. Reason: this variable is used to memorise the current location of the player which will be different for each player.
- Now every **aCurrentRoom** will be changed to **aPlayer.getCurrentRoom()**. (An accesseur getCurrentRoom() need to be added onto class Player)

```
/**
     * return the current room
     * @return this.aCurrentRoom
     */
    public Room getCurrentRoom(){
        return this.aCurrentRoom;
    }
```

- Problem: **getCurrentRoom()** is a value and cannot replace in any method where aCurrentRoom is a variable (aCurrentRoom can be both a value and a variable but aPlayer.getCurrentRoom cannot).
- Whatever method that use aCurrentRoom as a variable need to be changed (e.g: back and goRoom method)

⇒Add a modificateur **setCurrentRoom(paramètre)** of the current Room into class Player

```
/**
     * return the current room
     */
    public void setCurrentRoom(final Room pCurrentRoom){
        this.aCurrentRoom = pCurrentRoom;
    }
```

Class GameEngine

createRoom():

```
//this.aCurrentRoom = outside;//old code
this.aPlayer.setCurrentRoom(outside);
```

goRoom():

```
//this.aCurrentRoom = vNextRoom;//old code
this.aPlayer.setCurrentRoom(vNextRoom);
```

back():

```
//this.aPlayer.getCurrentRoom() = this.preRoom.peek();//old code
this.aPlayer.setCurrentRoom(this.preRoom.peek());
```

2. Stack<> preRoom

- For the same reason, **Stack<Room> preRoom** is also moved from GameEngine to Player.
- An accesseur **getPreRoom()** is added.
- **preRoom** is now replaced with **aPlayer.getPreRoom()**

Class Player:

```
/**
     * return the Stack preRoom of the player
     * this stack store the previous room that the player has passed
     */
    public Stack<Room> getPreRoom(){
        return this.preRoom;
    }
```

Class GameEngine - method back()

```
//if(this.preRoom().empty)//old code
if(this.aPlayer.getPreRoom().empty())

//this.preRoom().pop;//old code
this.aPlayer.getPreRoom().pop();
```

Class GameEngine - method goRoom()

```
//this.preRoom.push(currentRoom)//old code
this.aPlayer.getPreRoom().push(aPlayer.getCurrentRoom());
```

3. Weight

- Every player have a limited weight **aWeight** they can carry
- This variable will be initialised to be 20 kg for example, whenever the player take an item, this number will be decrease by how much that item weight (as well as increase whenever he drop or use an item)
- Once the weight of an item is more than the value of aWeight , they can no longer take that item

4. Inventory

- An HashMap **aInventory**, serve as an inventory to hold the item that the player take
- HashMap is used instead of Stack because it store objects and associate with an unique key (in this case the key are going to be the name of the item) which will be easier to access them later on (future command such as drop, take or use where the paramètre (or secondCommand) is the item)

5. Player's name

- Name of the player

6. To be added . . .

Class Player (to this point)

```
import java.util.Stack;
import java.util.HashMap;
/**
 * Class player
 *
 * @author Minh Quang TRAN
 */
public class Player
```

```
{
    private Room                    aCurrentRoom;
    private Stack<Room>             preRoom;
    private HashMap<String, Item>   aInventory;
    private String                  playerName;
    private int                     aWeight;
    /**
     * create class player
     */
    public Player()
    {
        this.aCurrentRoom =     null;
        this.preRoom =          new Stack<Room>();
        this.aInventory =       new HashMap<String, Item>();
        this.playerName =       "";
        this.aWeight =          23000;
    }

    /**
     * return the current room
     * @return this.aCurrentRoom
     */
    public Room getCurrentRoom(){
        return this.aCurrentRoom;
    }

    /**
     * modify the current room
     * @param pCurrentRoom - modificateur
     */
    public void setCurrentRoom(final Room pCurrentRoom){
        this.aCurrentRoom = pCurrentRoom;
    }

    /**
     * return the Stack preRoom of the player
     * this stack store the previous room that the player has passed
     * @return this.preRoom;
     */
    public Stack<Room> getPreRoom(){
        return this.preRoom;
    }
```

Exercice 7.30, 7.31: method take and drop

Methods that will be used by take() & drop()

Class CommandWords:

- 2 new commands "**take**" & "**drop**" added

```
private String[] aValidCommands = { "go", "help", "quit", "eat", "look", "back",
"test", "take", "drop"};
```

Class player:

- Whenever an item is taken or removed, the weight changes

⇒ An accesseur and a modificateur of the weight

```
/**
     * accesseur of the weight
     * @return this.aWeight
     */
    public int getWeight(){
        return this.aWeight;
    }
    /**modificateur of the weight
     * @return this.aWeight = pWeight
     */
    public int setWeight(final int pWeight){
        return this.aWeight += pWeight;
    }
```

**Some of the following code has been modified (or has been deleted completely) after ItemList had been created

HM aInventory

- New method **getInventoryInfo()** return the current Weight and all the Item taken
- This can be used to check whether or not the take() and drop() methods work or not
- This can then be printed by the look() method in class Room

```
/**
     * this method return information of the inventory
     * the item that has been put in and the amount of weight the player can
carry left
     * @return sItem
     */
    public String getInventoryInfo(){
        String sItem = "Inventory: ";
```

```
        Set<String> keys = aInventory.keySet();
        for(String item: keys){
            sItem += item + ", ";
        }
        if(sItem.equals("Inventory: ")){
            return "No item found!";
        }

        sItem += "\n" + "Weight left: " + this.aWeight;

        return sItem;
    }//getInventoryInfo()
```

(This method was shorten after ItemList had been created)

Method take()

Class Room

- Whenever an item is taken, it has to be removed from the room

⇒ New method **removeItem(param)**

```
/**
     * remove item is the reverse of add item
     * it removes the item from the room
     * this can be used for item that can be only taken once
     * @param item - item to be taken
     */
    public void removeItem(Item item){
        items.remove(item);
    }//removeItem
```

- For the take() method to work, we need another method that convert the user input to the item, from command to String (key), then to the item associated with that key

⇒**getRoomItem()** (was deleted after ItemList had been created)

- The method **get()** return the item that is associated with the key
- The method **containsKey()** return a boolean value true if the key exist in the hashmap, false otherwise

```
/**
 * return the item base on the input (name of the item)
 * @pCommand - name of the item
 * @return items.get(pCommand.getSecondWord())
 */
public Item getRoomItem(final Command pCommand){
    if(!this.items.containsKey(pCommand.getSecondWord())){
```

```
        return null;
    }else{
        return this.items.get(pCommand.getSecondWord());
    }
}
```

Class Player:

- New method **takeItem()** will put the item into the inventory. This method will be called by **take()** in class GameEngine. This procedure is similar to **addItem()** in class Room, except the paramètre is Command-typed

```
/**
    * takeItem add the item to the inventory
    * @param pCommand - item to be taken
    */
   public void takeItem(Command pCommand){
       Item pItem = aCurrentRoom.getItem(pCommand);
       this.aInventory.put(pCommand.getSecondWord(), pItem);
   }
```

method take () - Class GameEngine:

- This new command will require 2 command words, the second command is the name (or key) associate with the item
- If there isn't a second command, or the second command doesn't match the item of the room then a message will be printed to let the player know, the take() command will continue otherwise
- If the item weight more than what is left then the player is asked to remove some of the items from the inventory
- Every time an item is taken, it will be then added to the inventory and the variable aWeight of the player decreases by how much that item weighs. That item is also remove from the room

```
/**
 * take an item from the room
 * @param pCommand - name of the item to be taken
 */
```

```
private void take(Command pCommand){
    if(!pCommand.hasSecondWord()){
        this.aGui.println("Please specify which item you want to take");
        this.aGui.println("Type «look [item's name]» to take that item");
    }else{
        Item pItem = this.aPlayer.getCurrentRoom().getRoomItem(pCommand);
        //as mentioned, getItem return null if the key doesn't exist in the room
        if(pItem == null){
            this.aGui.println(pCommand.getSecondWord() + " might not be in this room");
        }else{
            if(this.aPlayer.getWeight() < pItem.getItemWeight()){
                this.aGui.println("Your inventory is full, please remove some item to
get more space");
            }else{
                aPlayer.takeItem(pCommand);
                this.aPlayer.getCurrentRoom().removeItem(pItem);

                this.aPlayer.setWeight(0 - pItem.getItemWeight(););
                this.aGui.println("this item has been taken");
                this.aGui.println(this.aPlayer.getInventoryInfo());
                //this.aGui.println("Item: "+ pItem.getItemNom());
            }
        }
        //<>
    }
}//take()
```

Method drop()

Class Player

- New method getPlayerItem, similar to getRoomItem (was deleted after ItemList had been created)

```
/**    same with getRoomItem, this method convert the command word
     * of the user's input to an item
     */
    public Item getPlayerItem(Command pCommand){
        if(!this.aInventory.containsKey(pCommand.getSecondWord())){
            return null;
        }else{
            return this.aInventory.get(pCommand.getSecondWord());
        }
    }
```

- For the same reason, **dropItem()** is created similar to **removeItem()** in class Room

```java
/**
     * dropItem remove the item to the inventory
     * @param pCommand - item to be dropped
     */
    public void dropItem(Command pCommand){
        //Item pItem = aCurrentRoom.getItem(pCommand);
        this.aInventory.remove(pCommand.getSecondWord());
    }
```

method drop() - Class GameEngine

- This command will require 2 command words, the second command is the name (or key) associate with the item
- If there isn't a second command, or the second command doesn't match the item of the inventory then a message will be printed to let the player know, the **drop()** command will continue otherwise
- Every time an item is dropped, it will be then added to the current room and the variable aWeight of the player increases by how much that item weighs.

```java
/**
 * drop an item from the inventory
 * @param vCommand - name of the item to be dropped
 */
private void drop(Command pCommand){
    if(!pCommand.hasSecondWord()){
        this.aGui.println("please specify which item you want to drop");
        this.aGui.println("type «drop [item's name]» to drop that item");
    }else{
        Item pItem = this.aPlayer.getPlayerItem(pCommand);

        if(pItem == null){
            this.aGui.println(pCommand.getSecondWord() + " haven't be taken yet ");
        }else{
            aPlayer.dropItem(pCommand);
            this.aPlayer.setWeight(pItem.getItemWeight());

            this.aPlayer.getCurrentRoom().addItem(pItem);

            this.aGui.println("this item has been dropped");
            this.aGui.println(this.aPlayer.getInventoryInfo());
            //this.aGui.println("Item: "+ pItem.getItemNom());
        }
    }
}
```

Exercice 31.1: New class ItemList

- ItemList object will be an HashMap that replace aInventory in class Player and items in class Room
- Reason: in the previous exercice, lots of additional methods are made specifically for either take or drop though we already have in Class Room. For example: 2 method addItem() and removeItem() can only be used with Room and not with the Player's inventory
- getRoomItem() & getPlayerItem() can be replaced with **object.getItemList().getItem(param)**

Class ItemList

- 2 new methods **addItemList()** and **removeItemList()**. In the future whenever we need to create a new method that adds (resp. remove) an item to the hashmap, these methods can be called

```
/**
 * add an Item to the hashmap (of player and room), similar to setExit
 * this method only need one paramètre because the key of the item
 * is the same with the item's name
 * @param item - item to be added
 */
public void addItemList(Item pItem){
    String s = pItem.getItemNom();
    aItemList.put(s, pItem);
}//addItem

/**
 * removeItem can be used to remove an item from the
 * inventory of the player or from a room
 * @param item - item to be taken or drop
 */
public void removeItemList(final String itemNom){
    aItemList.remove(itemNom);
}//removeItem
```

- getRoomItem() and getPlayerItem() can now be (consider to be) merged into **getItem()**

Making the paramètre type Command (instead of a String) was a mistake (will be mentioned in exercice 7.44). We can convert a Command to a String but not the other way around
New method hasItem() was also added on exercice 7.44 as well, even though it should have been done here

```
public Item getItem(final Command pCommand){
    if(!this.aItemList.containsKey(pCommand.getSecondWord())){
        return null;
    }else{
        return this.aItemList.get(pCommand.getSecondWord());
    }
}
```

- getItemString()(has been changed to getRoomsItem) was already created which returns all the items in a room and getInventoryInfo() which return all the items of the player's inventory.

⇒ New method **getItemString()** return all the item of the HashMap, to be called by those 2 methods mentioned

```
/**
     * this method return all of the item in HashMap
     * @return sItem
     */
    public String getItemString(){
        String sItem = "";
        Set<String> keys = aItemList.keySet();
        for(String item: keys){
            sItem += item + ", ";
        }
        if(sItem.equals("")){
            return "No item found!";
        }

        return sItem;
    }//getInventoryInfo()
```

(getInventoryInfo and getRoomsItem can just call this method)

- Now every `HashMap<String, Item>` has to be changed to ItemList. Some HashMap method such as put() and remove() are now replaced with addItem() and removeItem()

Class Player

- Constructeur et constructeur par défaut

```
private ItemList        aInventory;
this.aInventory =       new ItemList();
```

- Method **takeItem()** and dropItem()

```
public void takeItem(Command pCommand){
        //Item pItem = aCurrentRoom.getRoomItem(pCommand);//old code
        Item pItem = aCurrentRoom.getItemList().getItem(pCommand);

        //this.aInventory.put(pCommand.getSecondWord(), pItem);//old code
        this.aInventory.addItemList(pItem);
    }

public void dropItem(Command pCommand){
        //this.aInventory.remove(pCommand.getSecondWord());//old code
        this.aInventory.removeItemList(pCommand.getSecondWord());
    }*// dropItem() has been removed
```

- Accesseur **getItemList()**

```
    /**
     * accesseur of the inventory
     * @return this.aInventory
     */
    public ItemList getItemList(){
        return this.aInventory;
    }//getItemList
```

- **getInventoryInfo()** shorten

```
/**
     * this method return information of the inventory
     * the item that has been put in and the amount of weight the player can
carry left
     * @return sItem
     */
    public String getInventoryInfo(){
        String sItem = "Inventory: ";

        /**/sItem += aInventory.getItemString();
        sItem += "\n" + "Weight left: " + this.aWeight;

        return sItem;
    }//getInventoryInfo()
```

Class Room

- Constructeur et constructeur par défaut

```
//private HashMap<String, Item> items; //store items of a room
```

```
private ItemList aRoomsItem;

//this.items = new HashMap<String, Item>();
this.aRoomsItem = new ItemList();
```

- **addItem()** is shorten and removeItem() is removed

```
/**
     * add and Item to the room
     * @param item - item to be added to the room in the contructeur par défaut
     */
    public void addItem(Item item){
        this.aRoomsItem.addItemList(item);
    }//addItem

    /*has been removed
    public void removeItem(final String itemNom){
        this.aRoomsItem.removeItemList(itemNom);
    }*/
```

- **getRoomItem()** return single item from the room
- **getRoomItemString()** return all the Item in the form of a String (used to be getItemString)

```
/**
     * getItemString is already declared in itemlist
     * so the name is changed to getRoomItemString()
     */
    public String getRoomItemString(){
        String sItem = "Items in this room:";
        sItem += aRoomsItem.getItemString();
        return sItem;
    }//getItemString
/**
     * return the item based on the user's input
     * @param pCommand - user's input
     * @return this.aRoomsItem.getItem(pCommand)
     */
    public Item getRoomItem(final Command pCommand){
        return this.aRoomsItem.getItem(pCommand);
    }
```

- Accesseur **getItemList()**

```
/**
     * return the list of the room
     * @return this.aRoomsItem;
     */
    public ItemList getItemList(){
```

```
        return this.aRoomsItem;
    }//getItemList
```

Class GameEngine

- The 2 methods **removeItem()** in class Room and **dropItem()** in class Room in class Player have been removed. any method that call these two can replace them with **objet.getItemList().removeItemList(item key - item String)**
- I chose not to use this with the **addItem()** and **takeItem()** as they have Item-typed paramètre
- take method

```
//Item pItem = this.aPlayer.getCurrentRoom().getRoomItem(pCommand);//old
Item pItem = this.Player.getCurrentRoom().getItemList().getItem(pCommand);
```

```
//this.aPlayer.getCurrentRoom().removeItem(pCommand.getSecondWord());//old
this.aPlayer.getCurrentRoom().getItemList().removeItemList(pCommand.getSecondWord());
```

- drop method

```
//Item pItem = this.aPlayer.getPlayerItem(pCommand);//old
Item pItem = this.aPlayer.getItemList().getItem(pCommand);
```

```
//this.aPlayer.dropItem(pCommand);
this.aPlayer.getItemList().removeItemList(pCommand.getSecondWord());
```

Exercice 7.32, 7.33: Weight & Inventaire

These 2 exercice were already (accidentally) done while creating the class Player in exercise 7.29

Exercice 7.34: Magic cookie

Class GameEngine

method interpretCommand

- modified so that the eat method process the command

```
else if(vCommandWord.equals("eat")){
            //this.eat();//old
         this.eat(vCommand);
        }
```

method **eat()** was changed to have paramètre

- If there is no second word or that second word doesn't match with the cookie, a message is printed to let the user know
- Once the user use the cookie, he get an extra weight on his inventory

```
/**
     * eat
     * @param pCommand - what the user's gonna eat
     */
    private void eat(final Command pCommand){
        //this.aGui.println("You have eaten and you are not hungry any more");
        if(!pCommand.hasSecondWord()){
            this.aGui.println("eat what?");
        }else{
            Item pItem = this.aPlayer.getItemList().getItem(pCommand);
            if(pItem == null){
                this.aGui.println("You haven't taken this item yet");
                return;
            }

            if(pCommand.getSecondWord().equals("magic_cookie")){
                this.aPlayer.setWeight(7000 + pItem.getItemWeight());
                this.aPlayer.getItemList().removeItemList(pCommand.getSecondWord());

                this.aGui.println("You just ate the magic cookie");
                this.aGui.println("Now you have an extra 7kg on your inventory");

                this.aGui.println(aPlayer.getInventoryInfo());
            }else{
                this.aGui.println("You cannot eat that !!!");
            }
        }

    }//eat
```

Exercice 7.42: time limit

Class Player: Step

- New variable aStep added on Player to count the player's step that they have made
- **getStep()** allow us to know how many steps the player has made
- **addStep()** will be called by goRoom and back whenever it is used, in order to add

```
private int             aStep;
public int getStep(){
    return this.aStep;
}

public void addStep(){
    this.aStep ++;
}
```

Class GameEngine

- New variable aStepLimit (not added in Class Player because it's the same for every player)

```
private int             aStepLimit = 20;
```

- Another condition is added to the method **goRoom()**, **back()** and **teleport (use beamer)**. Whenever the step count reach the limit, print the message to let the player know

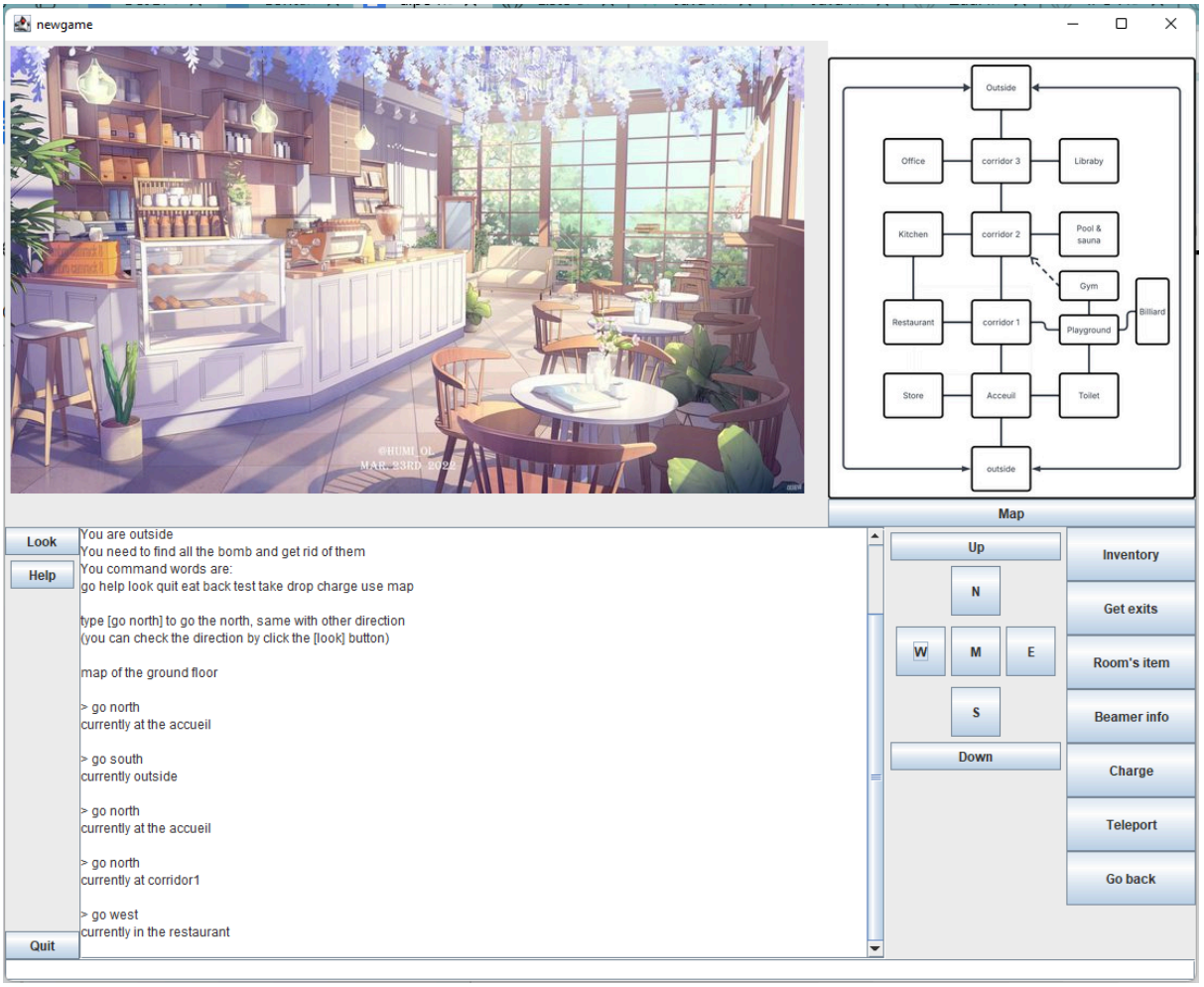
```
if(this.aPlayer.getStep() >= aStepLimit){
        this.aGui.println("You have taken too much time");
        this.aGui.println("The bombs exploded. You Lost!");
        return;
    }//if we get here, the the player's step count hasn't reached the limit yet
```

- At the end of **goRoom()**, **back()** and **teleport (use beamer)** add one to the step count

```
this.aPlayer.addStep();
```

Exercice 7.42.2: IHM

map



Exercice 7.43: trap door

Class Room

- Method containsValue return true if the room exist in the **HashMap<String, Room> exits** of the room, false otherwise

```
/**
     * detect whether or not the room (paramètre) exit in the HashMap "exits"
     * @param pBackRoom - the room to be "backed"
     * @return this.exits.containsValue(pBackRoom)
     */
    public boolean isExit(Room pBackRoom){
        return this.exits.containsValue(pBackRoom);
    }
```

Class GameEngine()

- Condition added in the back() method
- If the previous room is included in the **HashMap<String, Room>** of the current room then the player is allow to back, he isn't otherwise

```
if(!this.aPlayer.getCurrentRoom().isExit(this.aPlayer.getPreRoom().peek())){
    this.aGui.println("you just passed a trap door");
    this.aGui.println("You can only pass through one direction and not the other
way around");
    return;
}
```

Exercice 7.44: Beamer

New class Beamer created as a sorte of Item

- I added a variable aNTimeUsed which indicates how many times the user can use the beamer. This number decreases by 1 each use. I initialised this variable to 3 but it could be changed later on
- Accesseur and modificateur of the aChargedRoom
- Accesseur and modificateur of the aNTimeUsedLeft
- a boolean method that return true if the user can still use the beamer, false otherwise

```
public class Beamer extends Item
{
    private Room aChargedRoom;
    private int aNTimeUsedLeft;
    /**
     * constructeur par défaut
     */
    public Beamer(final String pItemNom, final String pItemDescription, final
int pItemWeight, final Room pChargedRoom, final int pNTimeUsedLeft){
        super(pItemNom, pItemDescription, pItemWeight);
        this.aChargedRoom = pChargedRoom;
        this.aNTimeUsed = pNTimeUsedLeft;
    }

    public void chargeRoom(final Room pChargedRoom){
        this.aChargedRoom = pChargedRoom;
    }

    public Room getChargedRoom(){
        return this.aChargedRoom;
    }

    public void setUse(int deltaTime){
        this.aNTimeUsedLeft += deltaTime;
    }//

    public int getTimeUsed(){
        return aNTimeUsedLeft;
    }//number of times left the player can use

    public boolean canBeUse(){
        return aNTimeUsedLeft > 0;
    }//return true if it is still usable, false otherwise
}
//<>
```

Class ItemList

- As mentioned in exercice 31.1, I created a new method hasItem() and modified the method getItem()
- Both of these methods have String-typed paramètre

```
/**
     * @return this.aItemList.containsKey(pItemNom)
     */
    public boolean hasItem(String pItemNom){
        return this.aItemList.containsKey(pItemNom);
    }//hasItem
/**
     * @return
     */
    public Item getItem(String pItemNom){
        if(!this.hasItem(pItemNom)){
            return null;
        }
        return this.aItemList.get(pItemNom);

    }
```

Class CommandWords

- 2 new command “charged” and “use” added

```
private String[] aValidCommands = {"charge", "use"};
```

Class GameEngine

method charge()

- The method first detect whether or not the beamer has been taken and whether or not it is still usable
- If the beamer hasn't been charged, a single command "charge" will do. If it has, the player have to type "charge fast" or click the button I added

```
/**
 * charge method allows the player to charge one room to the beamer
 * it can then be used to teleport using the use() method
 * @param pCommand - user's input
 */
private void charge(final Command pCommand){
    if(!this.aPlayer.getItemList().hasItem("beamer")){
        this.aGui.println("You haven't taken the beamer yet");
        return;
    }//print the message if the beamer hasn't been taken yet

    Beamer vBeamer = (Beamer)this.aPlayer.getItemList().getItem("beamer");
    //when we use getItem, it returns an Item
    //it has to be converted to a Beamer
    //new Beamer declared to hold that converted thingy

    if(!vBeamer.canBeUse()){
            this.aGui.println("You use the beamer too much!");
            this.aGui.println("It cannot be used anymore!");
            return;
        }//if the variable aNTimeUseLeft of the beamer is 0, it cannot be used

    if(!pCommand.hasSecondWord()){
        if(vBeamer.getChargedRoom() != null){
            this.aGui.println(*warning message*);
            t.a.p("Charged room:" + vBeamer.getChargedRoom().getDescription());
            this.aGui.println("type «charge fast» to charge");
        }else{
            vBeamer.chargeRoom(this.aPlayer.getCurrentRoom());
            this.aGui.println("This room has been charge");
            //this.aGui.println(vBeamer.getChargedRoom().getDescription());
        }
        return;
    }
    if(pCommand.getSecondWord().equals("fast")){
        vBeamer.chargeRoom(this.aPlayer.getCurrentRoom());
        this.aGui.println("This room has been charge");
        //this.aGui.println(vBeamer.getChargedRoom().getDescription());
    }else{
        this.aGui.println("type «charge fast» to charge");
    }

}
```

method use()

- I chose the method **use()** instead of fire like in the exercice because maybe a new special item (that can be used) can potentially be added to the game.
- The use **method()** will have a Command-typed paramètre, the second word of the command will the the item that the player choose to use
-

```
/**
 * use allows player to use an item
 * @param pCommand - the object to be used
 */
private void use(final Command pCommand){
    if(!pCommand.hasSecondWord()){
        this.aGui.println("Use what ? ");
        return;
    }

    if(pCommand.getSecondWord().equals("beamer")){
        if(!this.aPlayer.getItemList().hasItem("beamer")){
            this.aGui.println("You haven't taken the beamer yet");
            return;
        }//print the message if the beamer hasn't been taken yet

        Beamer vBeamer = (Beamer)this.aPlayer.getItemList().getItem("beamer");
        //convert beamer from an Item to a Beamer

        if(vBeamer.getChargedRoom() == null){
            this.aGui.println("The beamer haven't been charged yet!");
            return;
        }//the beamer hasn't been charged yet

        Room vNextRoom = vBeamer.getChargedRoom();

        if(vNextRoom.equals(aPlayer.getCurrentRoom())){
            t.a.p("You are already " + aPlayer.getCurrtRm().getDescr());
            return;
        }//the player is in the same room with the one is charged in the beamer


        vNextRoom.setExit("lối ngược lại", aPlayer.getCurrentRoom());
        //new exit set so that the player can go use back() command
        //now if the player do “back()”, the previous room will be added
        //to the

        this.aPlayer.getPreRoom().push(aPlayer.getCurrentRoom());
        this.aPlayer.setCurrentRoom(vNextRoom);
```

```
        vBeamer.chargeRoom(null);
        //remove the charged room after used

        vBeamer.setUse(-1);
        //decrease the number of time beamer can be used

        this.aGui.println("You just teleported to the charged room");
        this.aGui.println("You are " +
this.aPlayer.getCurrentRoom().getDescription());

        if(this.aPlayer.getCurrentRoom().getImageName() != null ){

this.aGui.showImage(this.aPlayer.getCurrentRoom().getImageName());
        }else{
            this.aGui.println("");
        }
    }else{
        *there will be more item to be used!!*
    }
}
```

*Step count has also been added

Exercice 7.46: TransporterRoom

(RoomRandomiser)

new tableau Room[] aTabRoom

```
//Exercice 18.5
aTabRoom = new Room[]{outside, accueil, corridor2, corridor3, boutique, restaurant,
                      kitchen, office, toilet, kid, billard, gym, pool, library};
```

New Class RoomRandomizer

- method **nextInt(int param)** generate a random (whole) number between 0 (included) and param (excluded);
  In my game it's [0,13]
- method findRandomRoom() will return a random Room the the tab

```
import java.util.Random;
public class RoomRandomizer
{
    private Random  aRandom;
    private Room[]  aTabRoom;

    public RoomRandomizer(final Room[] pTabRoom){
        this.aRandom = new Random();
        this.aTabRoom = pTabRoom;
    }

    public Room findRandomRoom(){
        int tabLength = aTabRoom.length;
        int random = aRandom.nextInt(tabLength);
        return aTabRoom[random];
    }
}
```

New Class TransporterRoom

```
public class TransporterRoom extends Room
{
    private RoomRandomizer  aRoomRandomizer;

    public TransporterRoom(final String pDescription, final String pImage, final
Room[] pTabRoom){
        super(pDescription, pImage);
        this.aRoomRandomizer = new RoomRandomizer(pTabRoom);
    }

    @Override public Room getExit(String direction){
        return aRoomRandomizer.findRandomRoom();
    }
}
```

method getExit() return a random room (no matter the String input)

III. Déclaration obligatoire anti-plagiat

Some methods of HashMap, Scanner, Random, etc I have looked up on the following site (consulted rather than copy & paste)
w3schools
GeeksforGeeks
Oracle